大数据应用案例

费子成　　苏州大学

E-mail: feizicheng@suda.edu.cn

个人主页：https://web.suda.edu.cn/feizicheng/





01

HDFS编程实践

HDFS常用命令

HDFS有很多shell命令，其中，fs命令可以说是HDFS最常用的命令。利用该命令可以查看HDFS文件系统的目录结构、上传和下载数据、创建文件等。该命令的用法为：

```
hadoop fs [genericOptions] [commandOptions]
```

备注：Hadoop中有三种Shell命令方式：

01

hadoop fs适用于任何不同的文件系统，比如本地文件系统和HDFS文件系统

02

hadoop dfs只能适用于HDFS文件系统

03

hdfs dfs跟hadoop dfs的命令作用一样，也只能适用于HDFS文件系统

HDFS常用命令

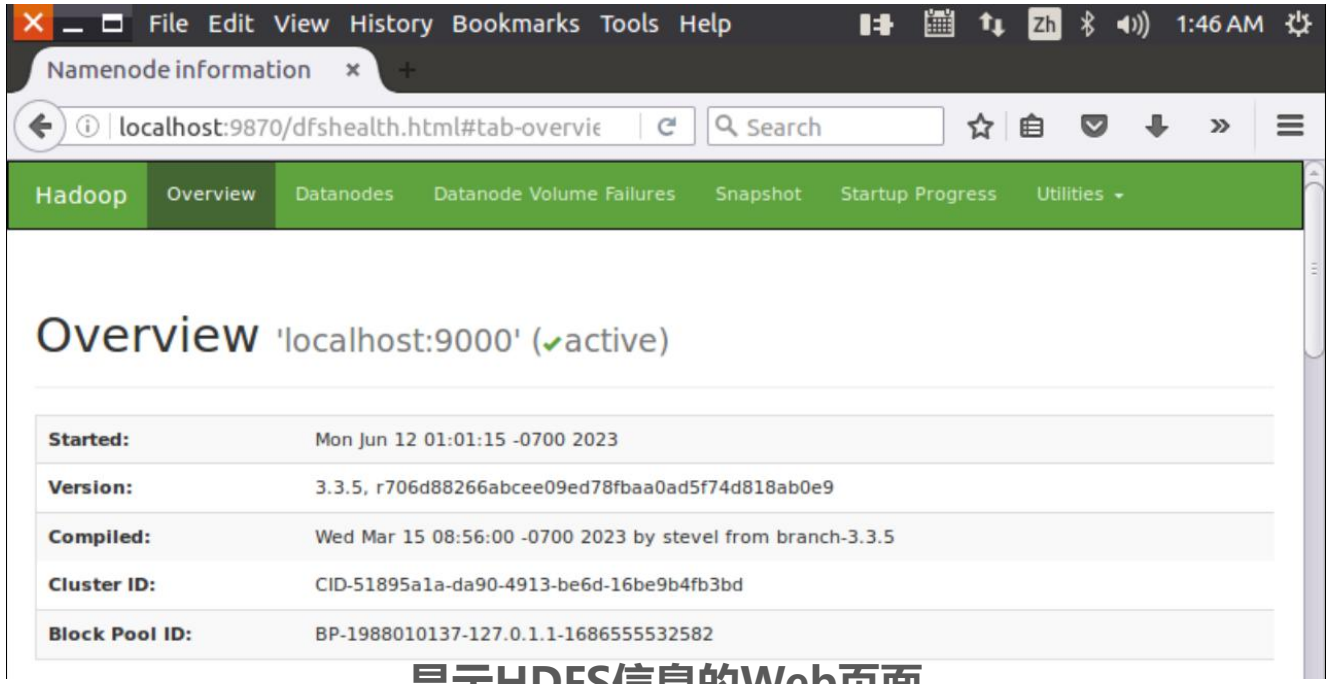
- hadoop fs -ls <path>。显示<path>指定的文件的详细信息。
- hadoop fs -ls -R <path>。ls命令的递归版本。
- hadoop fs -cat <path>。将<path>指定的文件的内容输出到标准输出（stdout）。
- hadoop fs -mkdir [-p] <paths>。创建<paths>指定的一个或多个文件夹，-p选项用于递归创建子文件夹。
- hadoop fs -copyFromLocal <localsrc> <dst>。将本地源文件<localsrc>复制到路径<dst>指定的文件或文件夹中。
- hadoop fs -copyToLocal [-ignorecrc] [-crc] <target> <localdst>。将目标文件<target>复制到本地文件或文件夹<localdst>中，可用-ignorecrc选项复制CRC校验失败的文件，使用-crc选项复制文件以及CRC信息。
- hadoop fs -cp <src> <dst>。将文件从源路径<src>复制到目标路径<dst>。
- hadoop fs -get [-ignorecrc] [-crc] <src> <localdst>。复制<src>指定的文件到本地文件系统<localdst>指定的文件或文件夹，可用-ignorecrc选项复制CRC校验失败的文件，使用-crc选项复制文件以及CRC信息。
- hadoop fs -rm <path>。删除<path>指定的文件，只删除非空目录和文件。
- hadoop fs -rm -r <path>。删除<path>指定的文件夹及其下的所有文件，-r选项表示递归删除子目录。



HDFS的Web页面

在配置好Hadoop集群之后，可以通过浏览器登录“http://[NameNodeIP]:9870”访问HDFS，其中，[NameNodeIP]表示名称节点的IP地址。

例如我们在本地机器上完成Hadoop伪分布式安装后，可以登录“http://localhost:9870”来查看文件系统信息（见图）。



显示HDFS信息的Web页面

HDFS常用Java API及应用实例

现在要执行的任务是：

假设在目录 “hdfs://localhost:9000/user/hadoop” 下面有几个文件

分别是 file1.txt、file2.txt、file3.txt、file4.abc和file5.abc 这里需要从该目录中筛选出所有后缀名不为“.abc”的文件。

对过滤之后的文件进行读取，并将这些文件的内容合并到文件 “hdfs://localhost:9000/user/hadoop/merge.txt”

HDFS常用Java API及应用实例

■ 编译运行程序

在开始编译运行程序之前，请一定确保Hadoop已经启动运行。

然后，要确保HDFS的“/user/hadoop”目录下已经存在file1.txt、file2.txt、file3.txt、file4.abc和file5.abc，每个文件里面有内容。

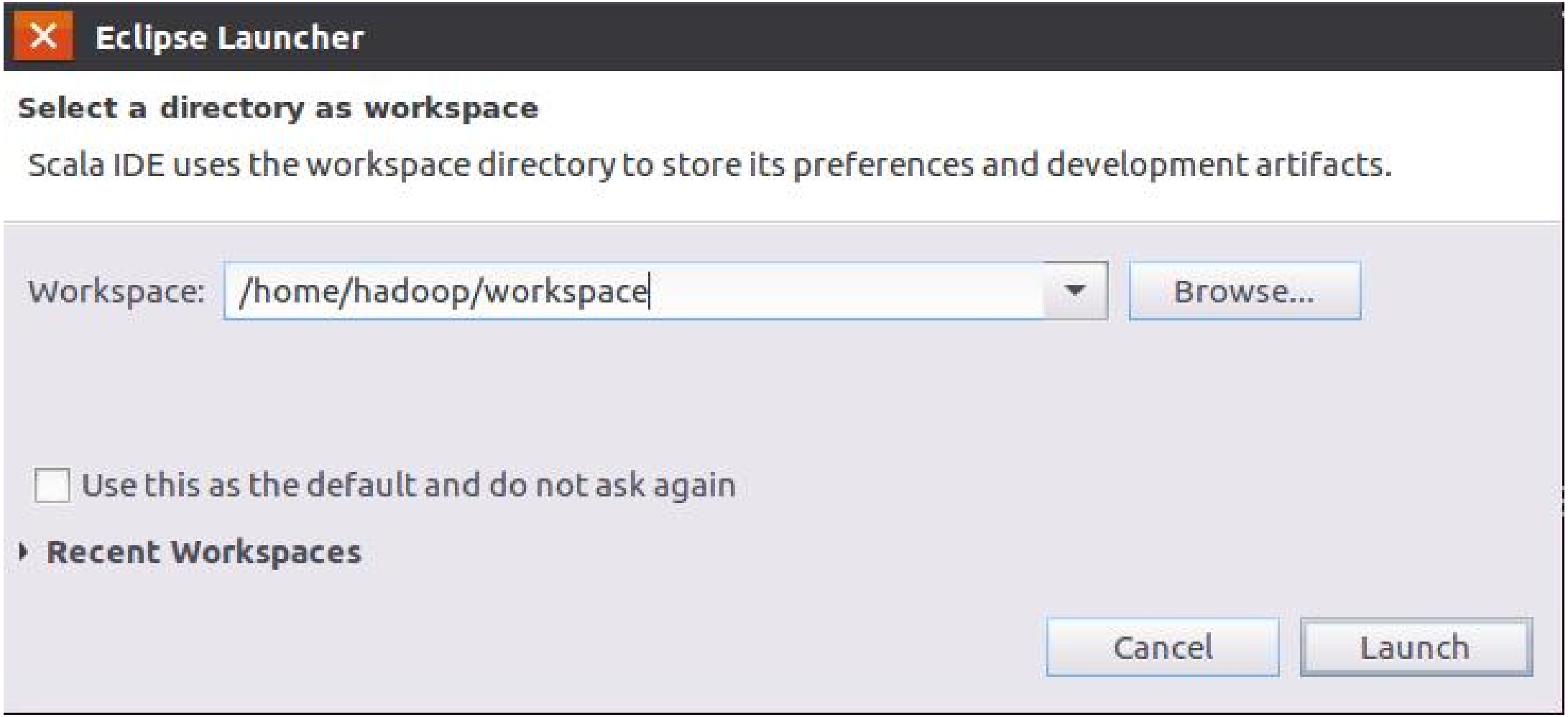
这里，假设文件内容如下表所示。

文件名称	文件内容
file1.txt	this is file1.txt
file2.txt	this is file2.txt
file3.txt	this is file3.txtfile
file4.abc	this is file4.abc
file5.abc	this is file5.abc

HDFS常用Java API及应用实例

- **在Eclipse中创建项目**

启动Eclipse。当Eclipse启动以后，会弹出如下图所示界面，提示设置工作空间（workspace）。



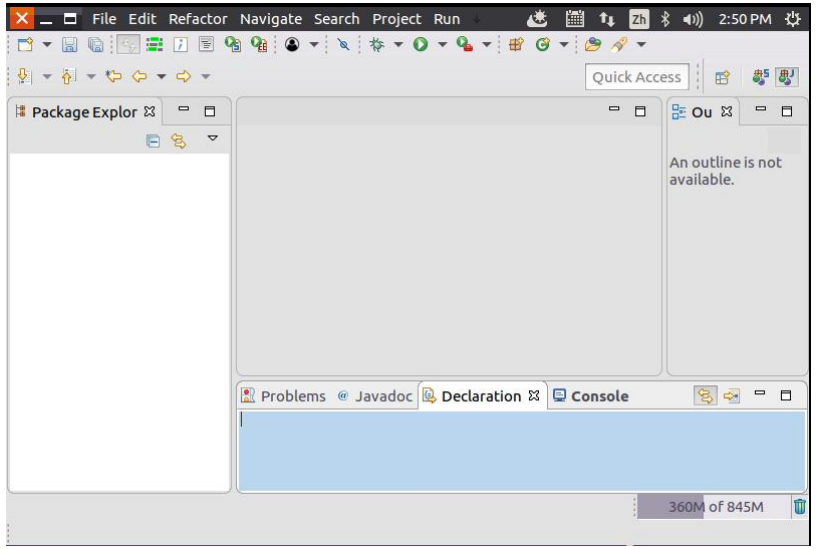
HDFS常用Java API及应用实例

- 在Eclipse中创建项目

可以直接采用默认的设置 “/home/hadoop/workspace”，点击“Launch”按钮。

可以看出，由于当前是采用hadoop用户登录了Linux系统，因此，默认的工作空间目录位于hadoop用户目录 “/home/hadoop”

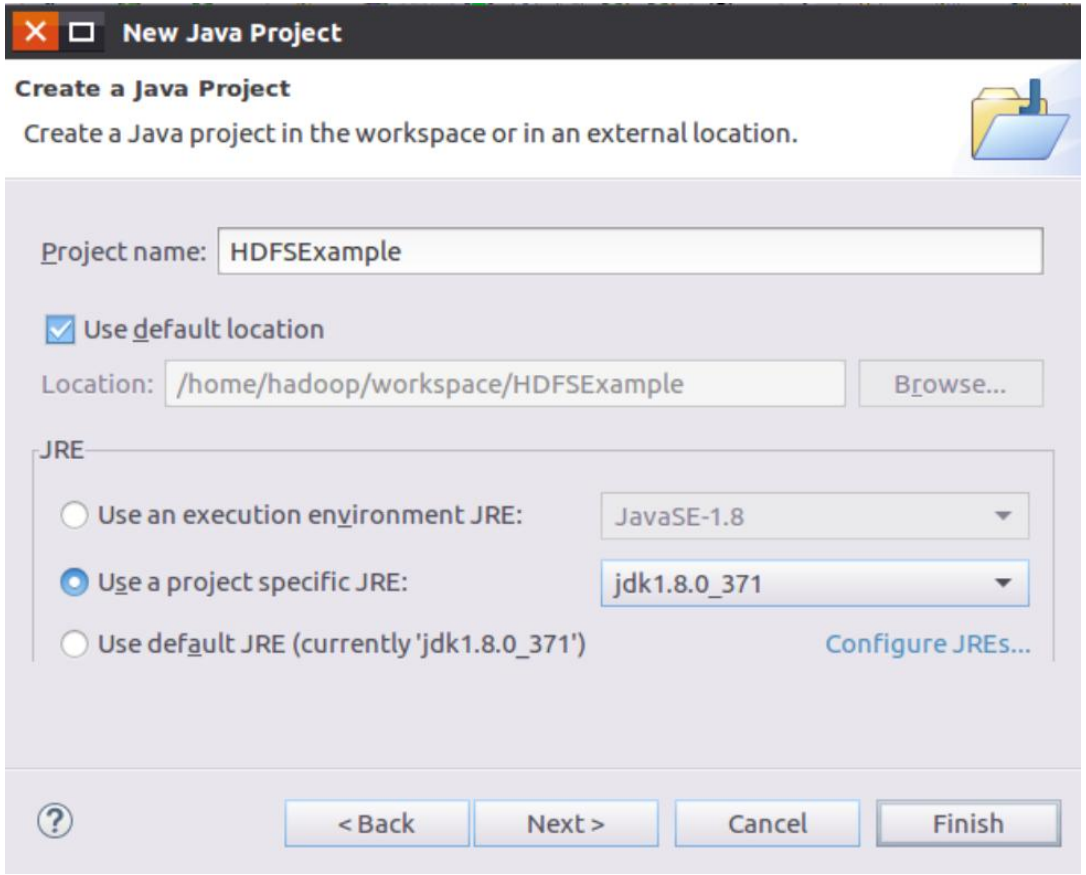
Eclipse启动以后，会呈现如下图所示的界面。



HDFS常用Java API及应用实例

- 在Eclipse中创建项目

选择“File->New->Java Project”菜单，开始创建一个Java工程，会弹出如下图所示界面。



HDFS常用Java API及应用实例

- 在Eclipse中创建项目

在“Project name”后面输入工程名称“HDFSExample”。

选中“Use default location”，
让这个Java工程的所有文件都保存到“/home/hadoop/workspace/HDFSExample”目录下。

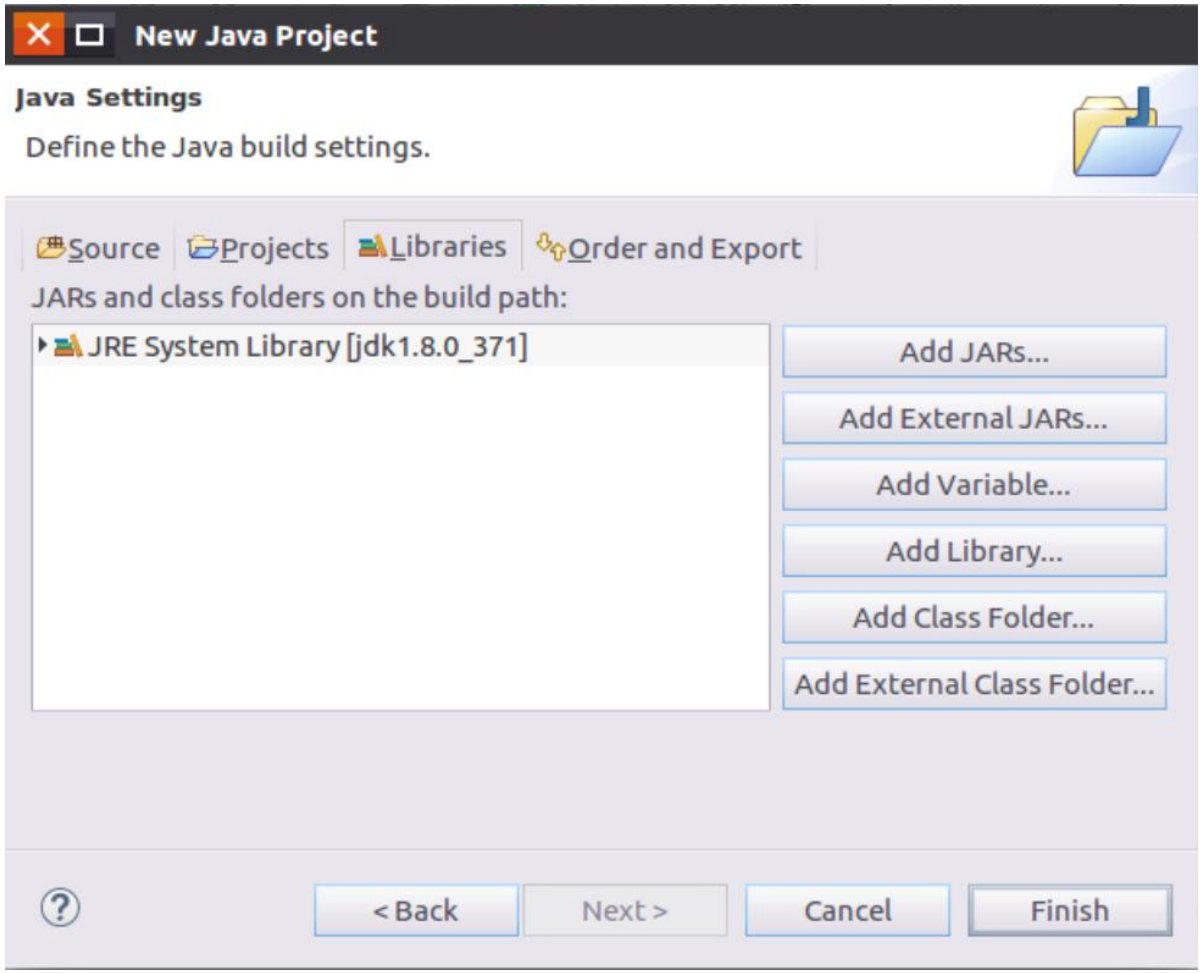
在“JRE”这个选项卡中，可以选择当前的Linux系统中已经安装好的JDK，比如jdk1.8.0_371。

然后，点击界面底部的“Next>”按钮，进入下一步的设置。

HDFS常用Java API及应用实例

- 为项目添加需要用到的JAR包

进入下一步的设置以后，会弹出如下图所示界面。



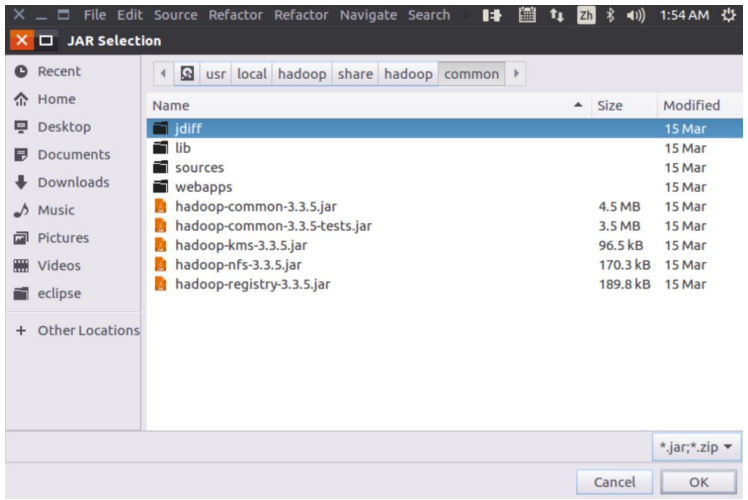
HDFS常用Java API及应用实例

■ 为项目添加需要用到的JAR包

需要在这个界面中加载该Java工程所需要用到的JAR包，这些JAR包中包含了可以访问HDFS的Java API。这些JAR包都位于Linux系统的Hadoop安装目录下。

对于本课程而言，就是在“/usr/local/hadoop/share/hadoop”目录下。点击界面中的“Libraries”选项卡。

点击界面右侧的“Add External JARs...”按钮，会弹出如下图所示界面。



HDFS常用Java API及应用实例

- 为项目添加需要用到的JAR包

在该界面中，上面的一排目录按钮（即“usr”、“local”、“hadoop”、“share”、“hadoop”和“common”），当点击某个目录按钮时，就会在下面列出该目录的内容。

为了编写一个能够与HDFS交互的Java应用程序，一般需要向Java工程中添加以下JAR包：

“/usr/local/hadoop/share/hadoop/common”目录下的所有JAR包，包括hadoop-common-3.3.5.jar、hadoop-common-3.3.5-tests.jar、haoop-nfs-3.3.5.jar、haoop-kms-3.3.5.jar和hadoop-registry-3.3.5.jar，注意，不包括目录jdiff、lib、sources和webapps；

“/usr/local/hadoop/share/hadoop/common/lib”目录下的所有JAR包；

“/usr/local/hadoop/share/hadoop/hdfs”目录下的所有JAR包，注意，不包括目录jdiff、lib、sources和webapps；

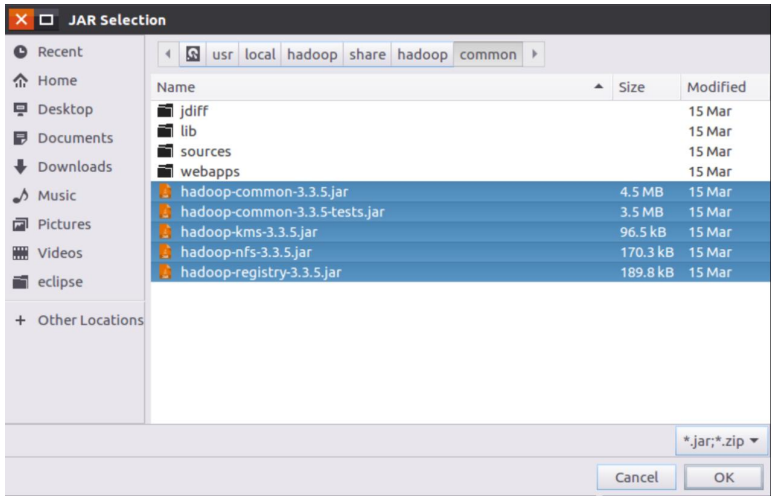
“/usr/local/hadoop/share/hadoop/hdfs/lib”目录下的所有JAR包。

HDFS常用Java API及应用实例

- 为项目添加需要用到的JAR包

如果要把“/usr/local/hadoop/share/hadoop/common”目录下的hadoop-common-3.3.5.jar、hadoop-common-3.3.5-tests.jar、haoop-nfs-3.3.5.jar、haoop-kms-3.3.5.jar和hadoop-registry-3.3.5.jar添加到当前的Java工程中：

可以在界面中点击目录按钮，进入到common目录，然后，界面会显示出common目录下的所有内容（如图所示）。

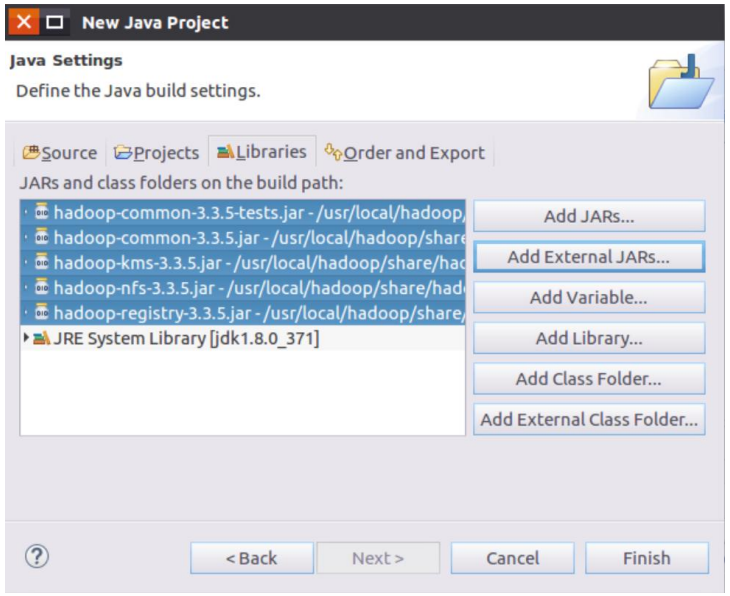


HDFS常用Java API及应用实例

- 为项目添加需要用到的JAR包

请在界面中用鼠标点击选中hadoop-common-3.3.5.jar、hadoop-common-3.3.5-tests.jar、haoop-nfs-3.3.5.jar、haoop-kms-3.3.5.jar和hadoop-registry-3.3.5.jar（不要选中目录jdiff、lib、sources和webapps），

然后点击界面右下角的“确定”按钮，就可以把这五个JAR包增加到当前Java工程中，出现的界面如图所示。



HDFS常用Java API及应用实例

- 为项目添加需要用到的JAR包

从这个界面中可以看出，hadoop-common-3.3.5.jar、hadoop-common-3.3.5-tests.jar、haoop-nfs-3.3.5.jar、haoop-kms-3.3.5.jar和hadoop-registry-3.3.5.jar已经被添加到当前Java工程中。

然后，按照类似的操作方法，可以再次点击“Add External JARs...”按钮，把剩余的其他JAR包都添加进来。

需要注意的是，当需要选中某个目录下的所有JAR包时，可以使用“Ctrl+A”组合键进行全选操作。

全部添加完毕以后，就可以点击界面右下角的“Finish”按钮，完成Java工程HDFSExample的创建。

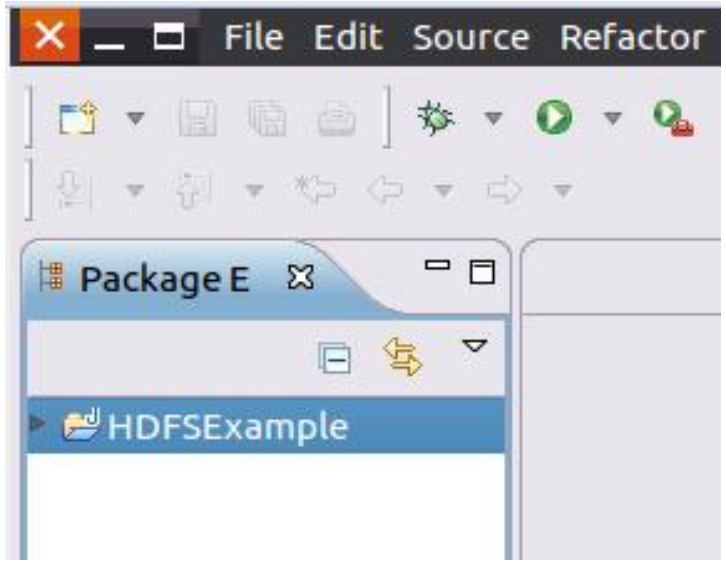
HDFS常用Java API及应用实例

- 编写Java应用程序

下面编写一个Java应用程序，用来检测HDFS中是否存在一个文件。

请在Eclipse工作界面左侧的“Package Explorer”面板中（如下图所示），找到刚才创建好的工程名称“HDFSExample”。

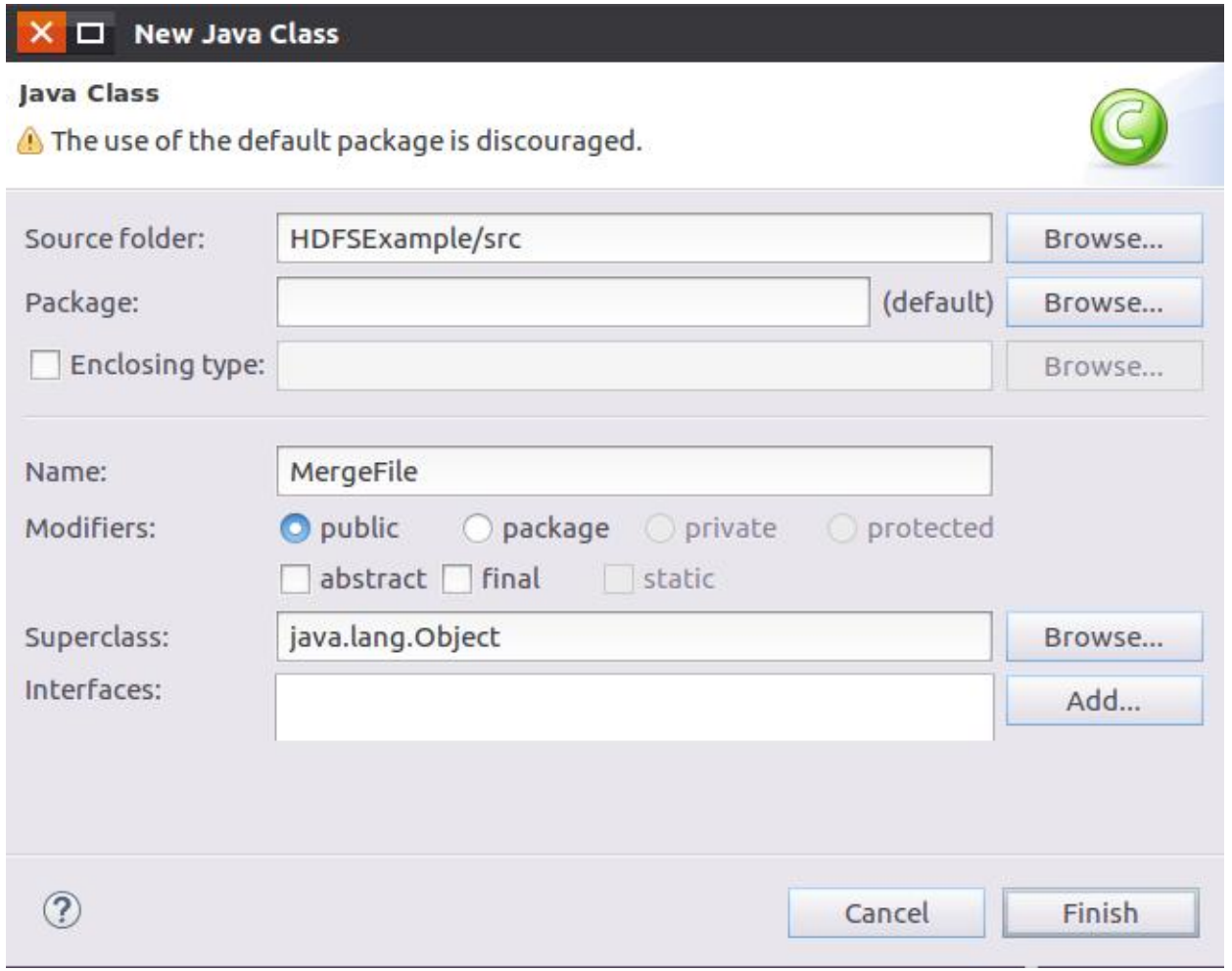
然后在该工程名称上点击鼠标右键，在弹出的菜单中选择“New Class”菜单。



HDFS常用Java API及应用实例

- 编写Java应用程序

选择“New　Class”菜单以后会出现如下图所示界面。

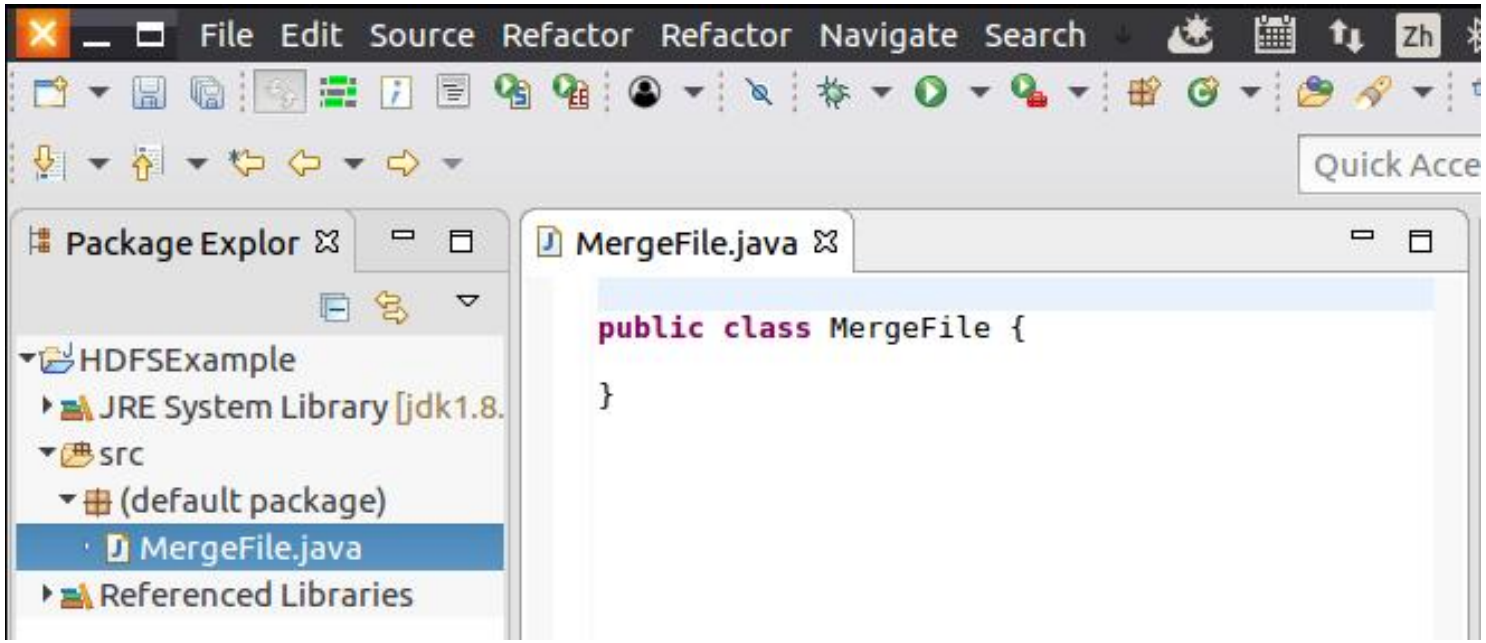


HDFS常用Java API及应用实例

- 编写Java应用程序

在该界面中，只需要在“Name”后面输入新建的Java类文件的名称，这里采用名称“MergeFile”，其他都可以采用默认设置。

然后，点击界面右下角“Finish”按钮，出现如下图所示界面。



HDFS常用Java API及应用实例

- **编写Java应用程序**

可以看出，Eclipse自动创建了一个名为“MergeFile.java”的源代码文件，请在该文件中输入以下代码：

```
import java.io.IOException;
import java.io.PrintStream;
import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.*;

/**
 * 过滤掉文件名满足特定条件的文件
 */
class MyPathFilter implements PathFilter {
    String reg = null;
    MyPathFilter(String reg) {
        this.reg = reg;
    }
    public boolean accept(Path path) {
                                                    if (!(path.toString().matches(reg)))
                                                                    return true;
                                                    return false;
                                    }
}
/***
 * 利用FSDataOutputStream和FSDataInputStream合并HDFS中的文件
 */
public class MergeFile {
                                    Path inputPath = null; //待合并的文件所在的目录的路径
                                    Path outputPath = null; //输出文件的路径
                                    public MergeFile(String input, String output) {
                                                    this.inputPath = new Path(input);
                                                    this.outputPath = new Path(output);
                                    }
                                    public void doMerge() throws IOException {
                                                    Configuration conf = new Configuration();
                                                    conf.set("fs.defaultFS","hdfs://localhost:9000");
        conf.set("fs.hdfs.impl","org.apache.hadoop.hdfs.DistributedFileSystem");
                                                    FileSystem fsSource = FileSystem.get(URI.create(inputPath.toString()), conf);
                                                    FileSystem fsDst = FileSystem.get(URI.create(outputPath.toString()), conf);
                                                                                    //下面过滤掉输入目录中后缀为.abc的文件
                                                    FileStatus[] sourceStatus = fsSource.listStatus(inputPath,
                                                                                    new MyPathFilter(".*\\.abc"));
                                                    FSDataOutputStream fsdos = fsDst.create(outputPath);
                                                    PrintStream ps = new PrintStream(System.out);
                                                    //下面分别读取过滤之后的每个文件的内容，并输出到同一个文件中
                                                    for (FileStatus sta : sourceStatus) {
                                                                    //下面打印后缀不为.abc的文件的路径、文件大小
                                                                    System.out.print("路径：" + sta.getPath() + "    文件大小：" + sta.getLen()
                                                                                                                    + "   权限：" + sta.getPermission() + "   内容：");
                                                                    FSDataInputStream fsdis = fsSource.open(sta.getPath());
                                                                    byte[] data = new byte[1024];
                                                                    int read = -1;

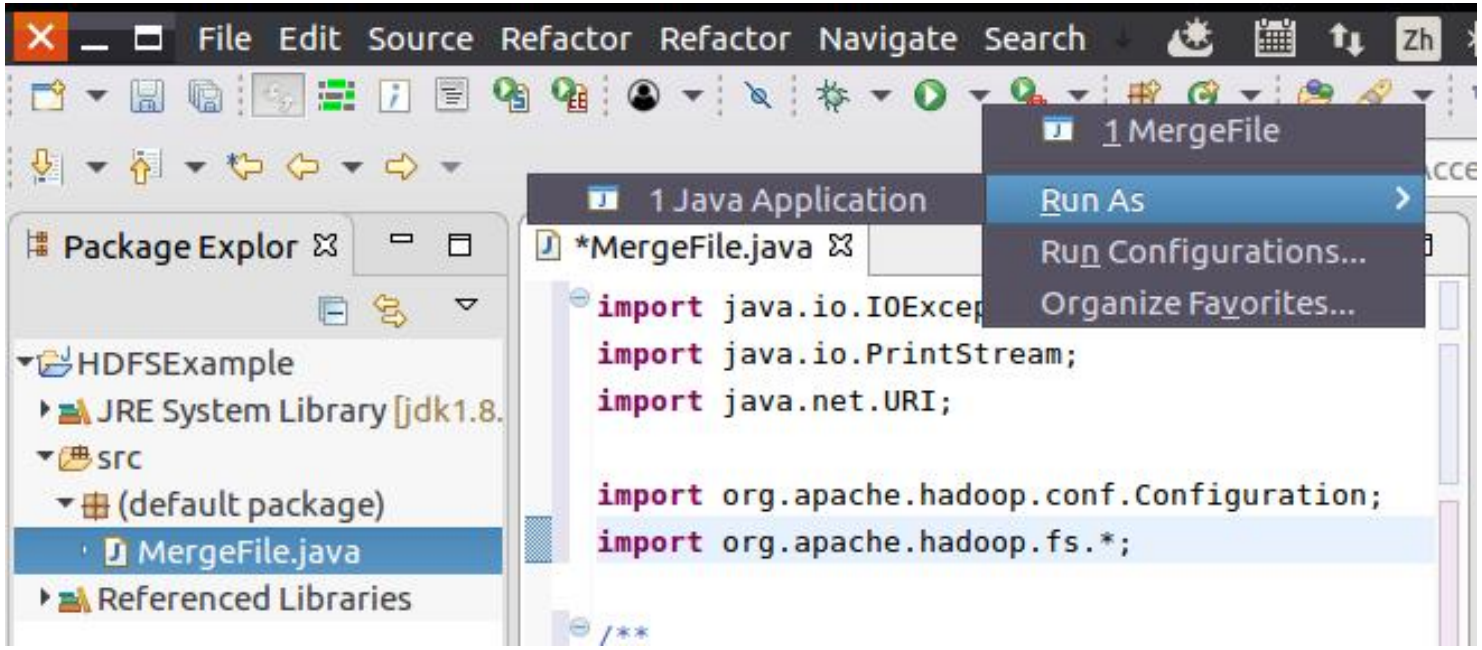
                                                                    while ((read = fsdis.read(data)) > 0) {
                                                                                    ps.write(data, 0, read);
                                                                                    fsdos.write(data, 0, read);
                                                                    }
                                                                    fsdis.close();
                                                    }
                                                    ps.close();
                                                    fsdos.close();
                                    }
                                    public static void main(String[] args) throws IOException {
                                                    MergeFile merge = new MergeFile(
                                                                                    "hdfs://localhost:9000/user/hadoop/",
                                                                                    "hdfs://localhost:9000/user/hadoop/merge.txt");
                                                    merge.doMerge();
                                    }
}
```

HDFS常用Java API及应用实例

- 编译运行程序

现在就可以编译运行上面编写的代码。可以直接点击Eclipse工作界面上部的运行程序的快捷按钮。

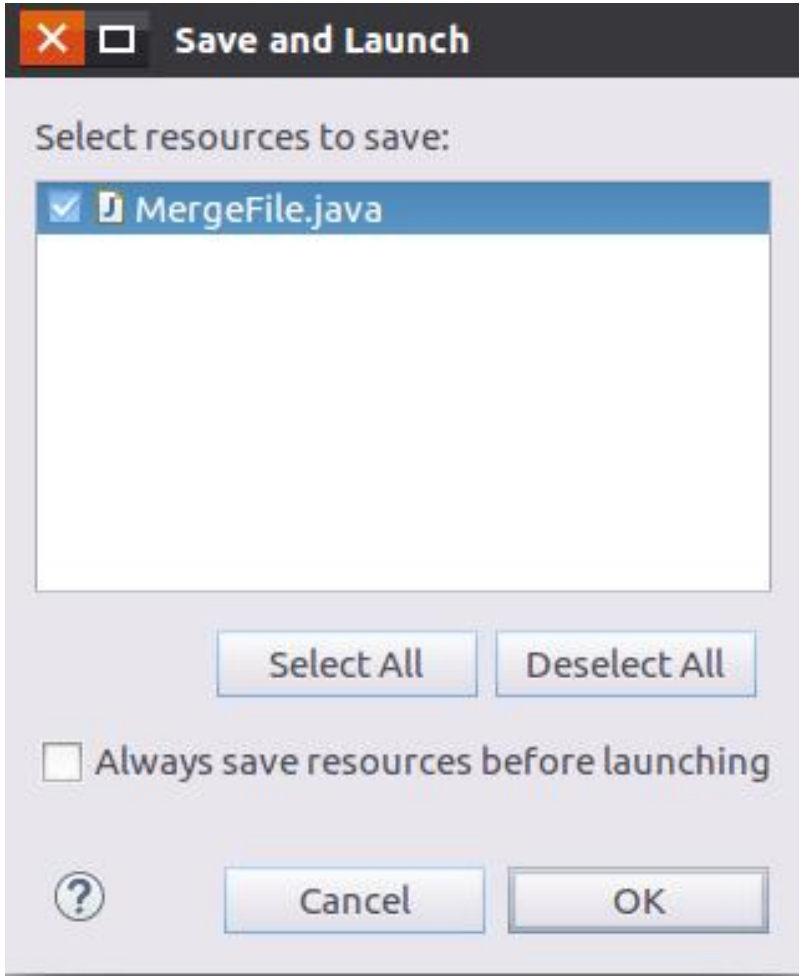
当把鼠标移动到该按钮上时，在弹出的菜单中选择“Run As”，继续在弹出来的菜单中选择“Java Application”，如下图所示。



HDFS常用Java API及应用实例

- **编译运行程序**

然后，会弹出如下图所示界面。

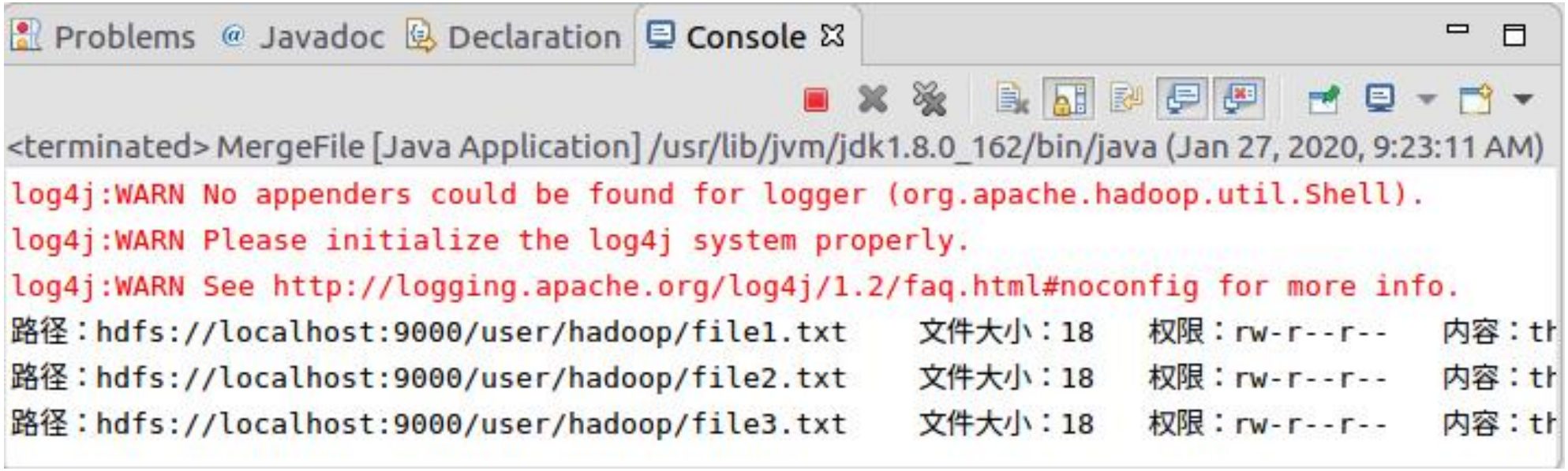


HDFS常用Java API及应用实例

- 编译运行程序

在该界面中，点击界面右下角的“OK”按钮，开始运行程序。程序运行结束后，会在底部的“Console”面板中显示运行结果信息（如下图所示）。

同时，“Console”面板中还会显示一些类似“log4j:WARN...”的警告信息，可以不用理会。



HDFS常用Java API及应用实例

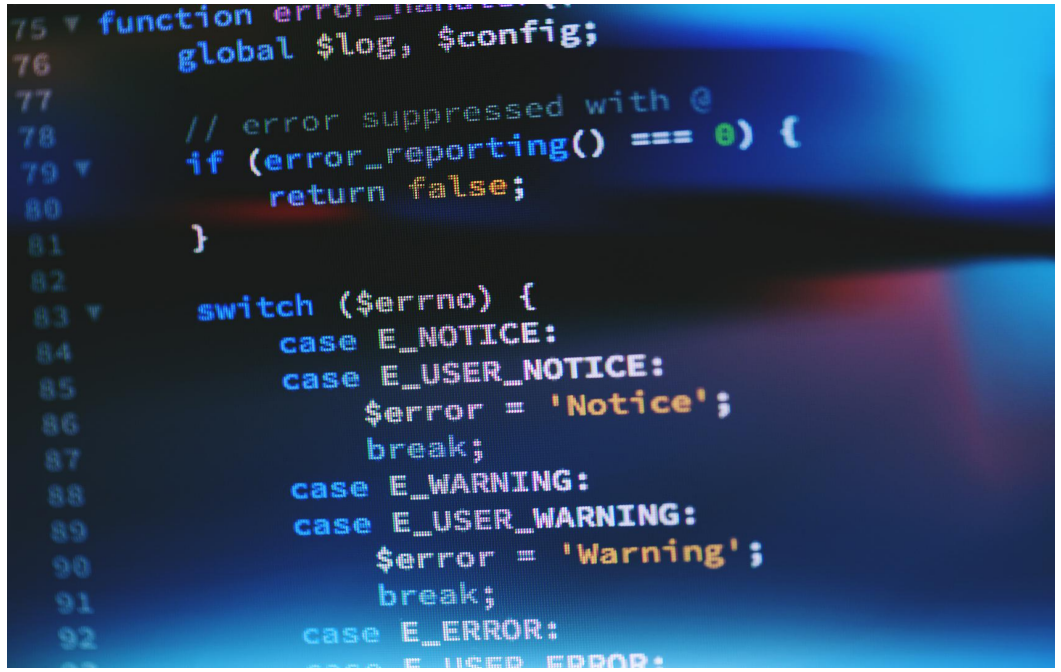
- 编译运行程序

如果程序运行成功，这时，可以到HDFS中查看生成的merge.txt文件，比如，可以在Linux终端中执行如下命令：

```
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -ls /user/hadoop
$ ./bin/hdfs dfs -cat /user/hadoop/merge.txt
```

可以看到如下结果：

```
this is file1.txt
this is file2.txt
this is file3.txt
```



HDFS常用Java API及应用实例

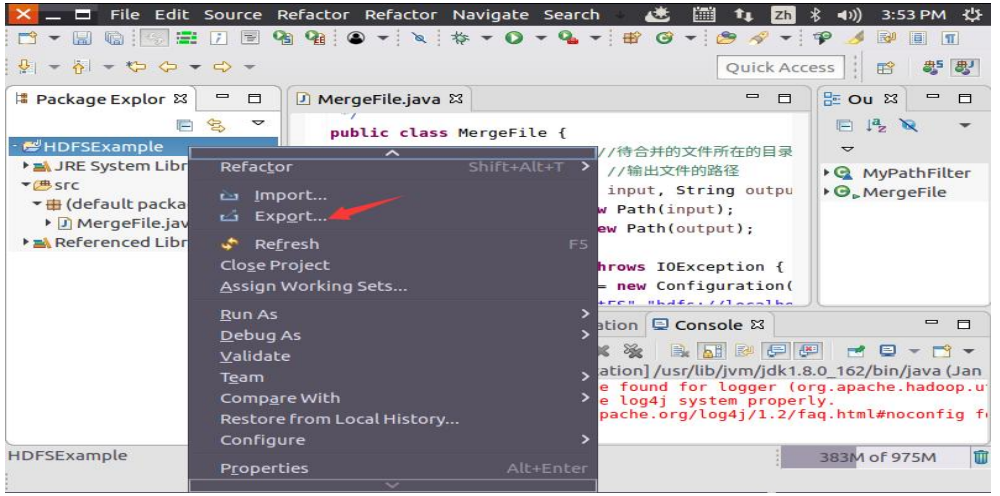
■ 应用程序的部署

下面介绍如何把Java应用程序生成JAR包，部署到Hadoop平台上运行。

在Hadoop安装目录下新建一个名称为myapp的目录，用来存放我们自己编写的Hadoop应用程序，可以在Linux的终端中执行如下命令：

```
$ cd /usr/local/hadoop
$ mkdir myapp
```

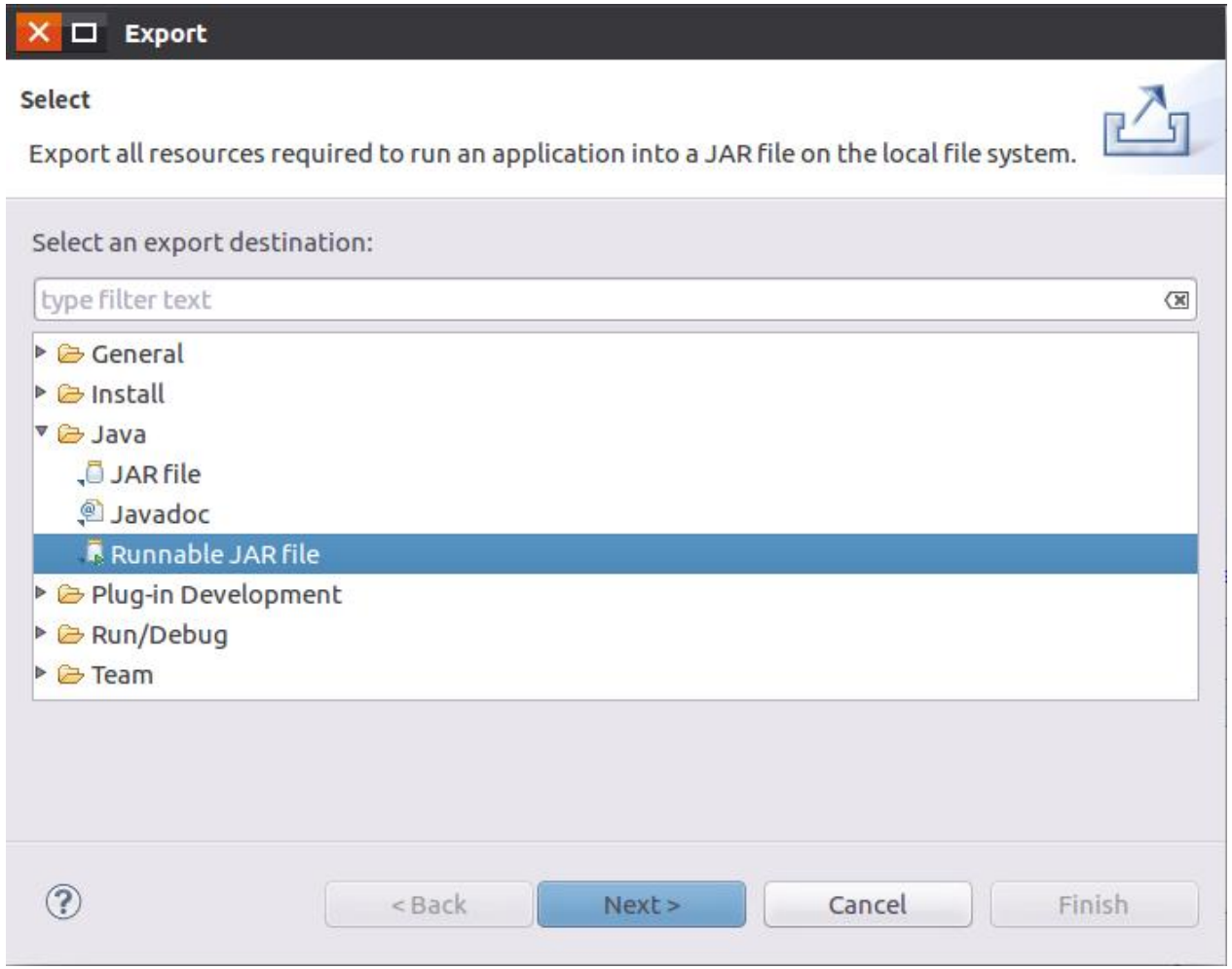
请在Eclipse工作界面左侧的“Package Explorer”面板中，在工程名称“HDFSExample”上点击鼠标右键，在弹出的菜单中选择“Export”，如图所示。



HDFS常用Java API及应用实例

- **应用程序的部署**

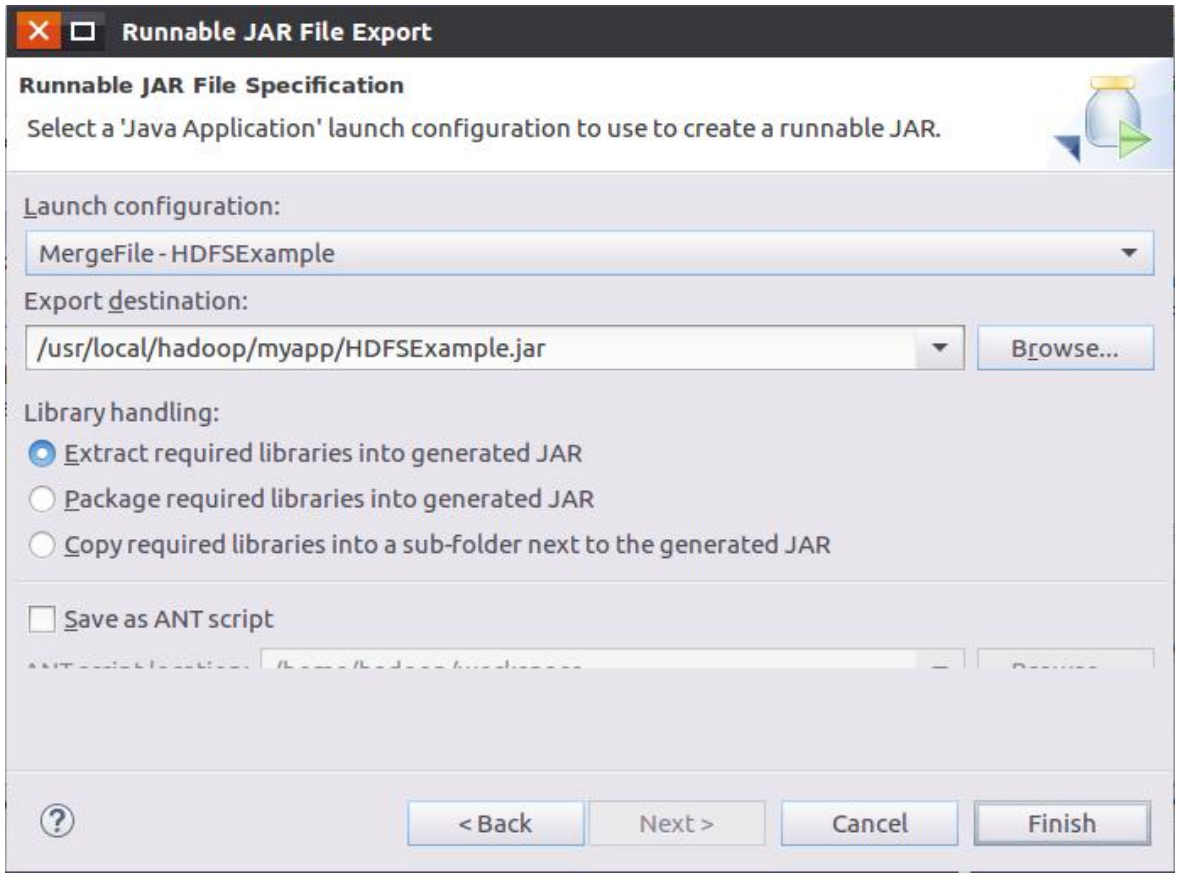
然后，会弹出如下图所示界面。



HDFS常用Java API及应用实例

- 应用程序的部署

在该界面中，选择“Runnable JAR file”，然后，点击“Next>”按钮，弹出如下图所示界面。



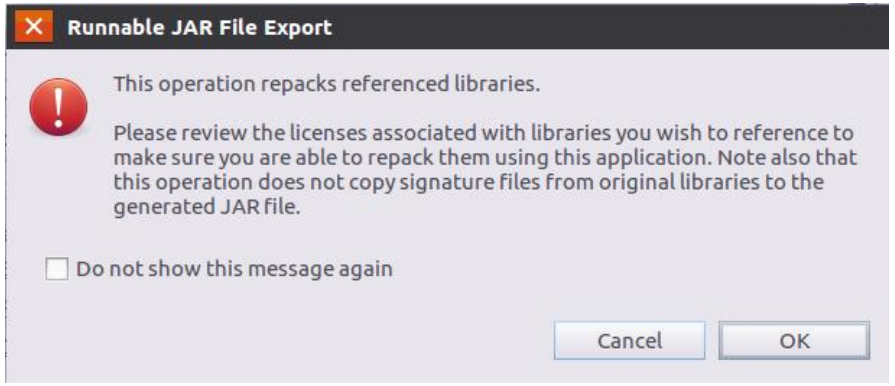
HDFS常用Java API及应用实例

■ 应用程序的部署

在该界面中，“Launch configuration”用于设置生成的JAR包被部署启动时运行的主类，需要在下拉列表中选择刚才配置的类“MergeFile-HDFSExample”。

在“Export destination”中需要设置JAR包要输出保存到哪个目录，比如，这里设置为“/usr/local/hadoop/myapp/HDFSExample.jar”。在“Library handling”下面选择“Extract required libraries into generated JAR”。

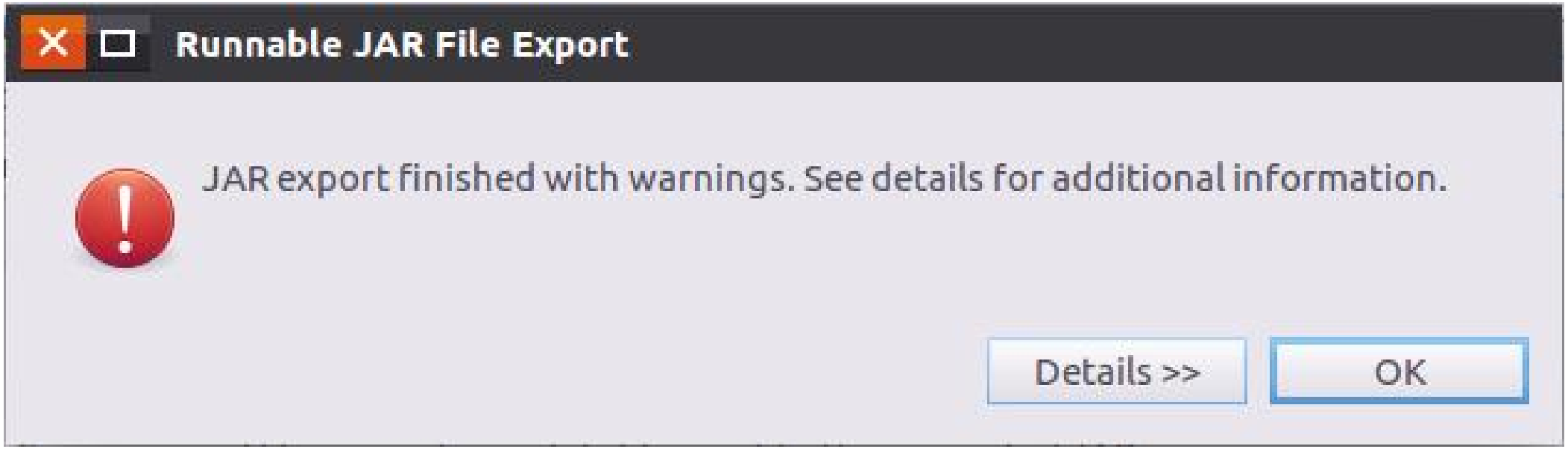
然后，点击“Finish”按钮，会出现如下图所示界面。



HDFS常用Java API及应用实例

- **应用程序的部署**

可以忽略该界面的信息，直接点击界面右下角的“OK”按钮，启动打包过程。打包过程结束后，会出现一个警告信息界面，如下图所示。



HDFS常用Java API及应用实例

- 应用程序的部署

可以忽略该界面的信息，直接点击界面右下角的“OK”按钮。至此，已经顺利把HDFSExample工程打包生成了HDFSExample.jar。

可以到Linux系统中查看一下生成的HDFSExample.jar文件，可以在Linux的终端中执行如下命令：

```
$ cd /usr/local/hadoop/myapp
$ ls
```

可以看到，“/usr/local/hadoop/myapp”目录下已经存在一个HDFSExample.jar文件。

HDFS常用Java API及应用实例

- 应用程序的部署

由于之前已经运行过一次程序，已经生成了merge.txt，因此，需要首先执行如下命令删除该文件：

```
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -rm /user/hadoop/merge.txt
```

现在，就可以在Linux系统中，使用hadoop jar命令运行程序，命令如下：

```
$ cd /usr/local/hadoop
$ ./bin/hadoop jar ./myapp/HDFSExample.jar
```

HDFS常用Java API及应用实例

- 应用程序的部署

上面程序执行结束以后，可以到HDFS中查看生成的merge.txt文件，比如，可以在Linux终端中执行如下命令：

```
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -ls /user/hadoop
$ ./bin/hdfs dfs -cat /user/hadoop/merge.txt
```

可以看到如下结果：

```
this is file1.txt
this is file2.txt
this is file3.txt
```

02

HBase编程实践

HBase的安装与配置

HBase安装

01

下载安装包hbase-2.5.4-bin.tar.gz

02

解压安装包hbase-2.5.4-bin.tar.gz至路径/usr/local

03

配置系统环境,将hbase下的bin目录添加到系统的path中

备注：前面课程中，安装完Hadoop时，只包含HDFS和MapReduce等核心组件，并不包含HBase，因此，HBase需要单独安装。

HBase的安装与配置

- HBase配置

单机模式 | 伪分布式模式 | 分布式模式

以下先决条件很重要，比如没有配置JAVA_HOME环境变量，就会报错。

JDK | Hadoop（单机模式不需要，伪分布式模式和分布式模式需要） | SSH

启动关闭Hadoop和HBase的顺序一定是：

启动Hadoop—>启动HBase—>关闭HBase—>关闭Hadoop

HBASE_MANAGES_ZK=true，则由HBase自己管理Zookeeper；
否则，启动独立的Zookeeper。
建议：单机版HBase，使用自带Zookeeper；集群安装HBase则采用单独Zookeeper集群。

HBase常用Shell命令

- HBase配置

create：创建表

list：列出HBase中所有的表信息

例子1：创建一个表，该表名称为tempTable，包含3个列族f1，f2和f3

```
hbase(main):002:0> create 'tempTable', 'f1', 'f2', 'f3'
0 row(s) in 1.3560 seconds

hbase(main):003:0> list
TABLE
tempTable
testTable
wordcount
3 row(s) in 0.0350 seconds
```

备注：后面的例子都在此基础上继续操作。

HBase常用Shell命令

- HBase配置

put：向表、行、列指定的单元格添加数据； 一次只能为一个表的一行数据的一个列添加一个数据

scan：浏览表的相关信息

例子2：继续向表tempTable中的第r1行、第“f1:c1”列，添加数据值为“hello,dblab”

```
hbase(main):005:0> put 'tempTable', 'r1', 'f1:c1', 'hello, dblab'
0 row(s) in 0.0240 seconds

hbase(main):006:0> scan 'tempTable'
ROW                           COLUMN+CELL
 r1                           column=f1:c1, timestamp=1430036599391, value=hello, dblab
1 row(s) in 0.0160 seconds
```

在添加数据时，HBase会自动为添加的数据添加一个时间戳，当然，也可以在添加数据时人工指定时间戳的值

HBase常用Shell命令

- HBase配置

get：通过表名、行、列、时间戳、时间范围和版本号来获得相应单元格的值

例子3：

（1）从tempTable中，获取第r1行、第“f1:c1”列的值

（2）从tempTable中，获取第r1行、第“f1:c3”列的值

备注：f1是列族，c1和c3都是列

```
hbase(main):012:0> get 'tempTable', 'r1', {COLUMN=>'f1:c1'}
COLUMN                          CELL
 f1:c1                          timestamp=1430036599391, value=hello, dblab
1 row(s) in 0.0090 seconds

hbase(main):013:0> get 'tempTable', 'r1', {COLUMN=>'f1:c3'}
COLUMN                          CELL
0 row(s) in 0.0030 seconds
```

从运行结果可以看出： tempTable中第r1行、第“f1:c3”列的值当前不存在

HBase常用Shell命令

- HBase配置

enable/disable：使表有效或无效

drop：删除表

例子4：使表tempTable无效、删除该表

```
hbase(main):016:0> disable 'tempTable'
0 row(s) in 1.3720 seconds

hbase(main):017:0> drop 'tempTable'
0 row(s) in 1.1350 seconds

hbase(main):018:0> list
TABLE
testTable
wordcount
2 row(s) in 0.0370 seconds
```

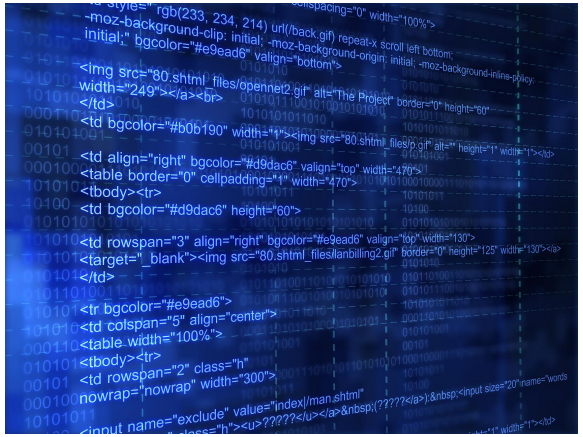
HBase常用Java API及应用实例

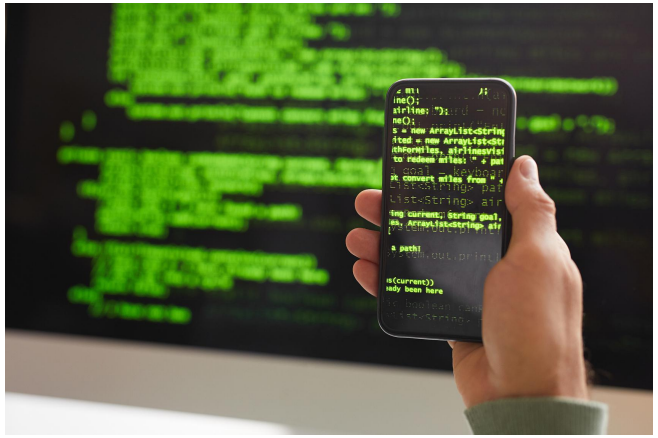
采用Eclipse开发工具编写HBase程序

HBase是Java编写的，它的原生的API也是Java开发的，不过，可以使用Java或其他语言调用API来访问HBase。

备注：使用的HBase版本号为2.5.4







HBase常用Java API及应用实例

- **任务要求：创建表、插入数据、浏览数据**

创建一个学生信息表，用来存储学生姓名（姓名作为行键，并且假设姓名不会重复）以及考试成绩，其中，考试成绩是一个列族，分别存储了各个科目的考试成绩。逻辑视图如表所示。

name	score		
	English	Math	Computer

学生信息表的表结构

name	score		
	English	Math	Computer
zhangsan	69	86	77
lisi	55	100	88

需要添加的数据

HBase常用Java API及应用实例

- **任务要求：创建表、插入数据、浏览数据**

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.hbase.*;
import org.apache.hadoop.hbase.client.*;
import org.apache.hadoop.hbase.util.Bytes;
import java.io.IOException;
public class Chapter4{
    public static Configuration configuration;
    public static Connection connection;
    public static Admin admin;

    public static void main(String[] args)throws IOException{
        createTable("student",new String[]{"score"});
        insertData("student","zhangsan","score","English","69");
        insertData("student","zhangsan","score","Math","86");
        insertData("student","zhangsan","score","Computer","77");
        getData("student", "zhangsan", "score", "English");
    }
……
public static void init(){……}//建立连接
public static void close(){……}//关闭连接
public static void createTable(){……}//创建表
public static void insertData() {……}//插入数据
public static void getData{……}//浏览数据
}
```



HBase常用Java API及应用实例

- 建立连接，关闭连接

```
//建立连接
    public static void init(){
        configuration  = HBaseConfiguration.create();
        configuration.set("hbase.rootdir","hdfs://localhost:9000/hbase");
        try{
            connection = ConnectionFactory.createConnection(configuration);
            admin = connection.getAdmin();
        }catch (IOException e){
            e.printStackTrace();
        }
    }
```

备注：hbase-site.xml

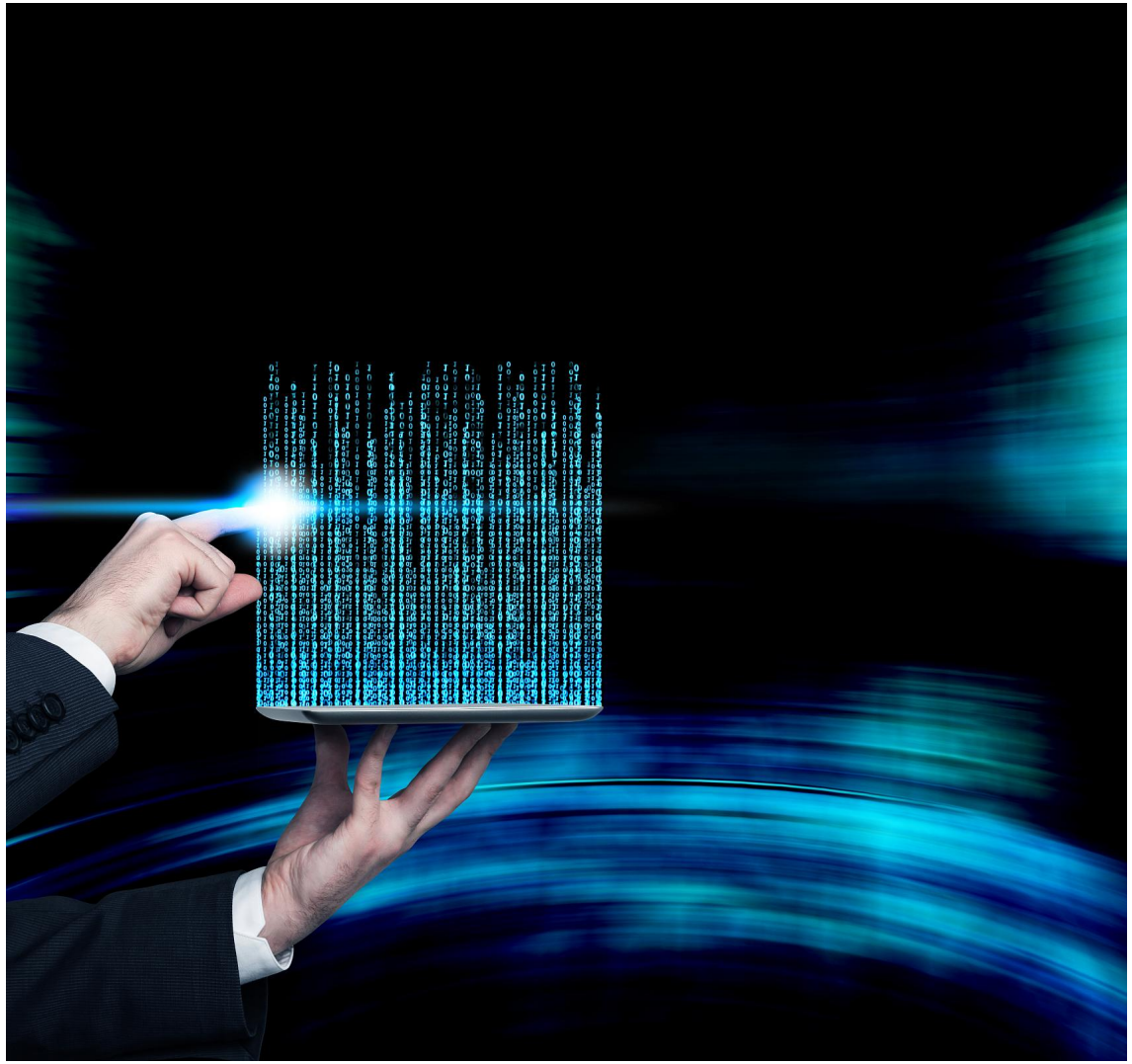
```
<configuration>
<property>
<name>hbase.rootdir</name>
<value>hdfs://localhost:9000/hbase</value>
</property>
</configuration>
```

（单机版）file:///DIRECTORY/hbase

HBase常用Java API及应用实例

- 建立连接，关闭连接

```
//关闭连接
    public static void close(){
        try{
            if(admin != null){
                admin.close();
            }
            if(null != connection){
                connection.close();
            }
        }catch (IOException e){
            e.printStackTrace();
        }
    }
```



HBase常用Java API及应用实例

■ 创建表

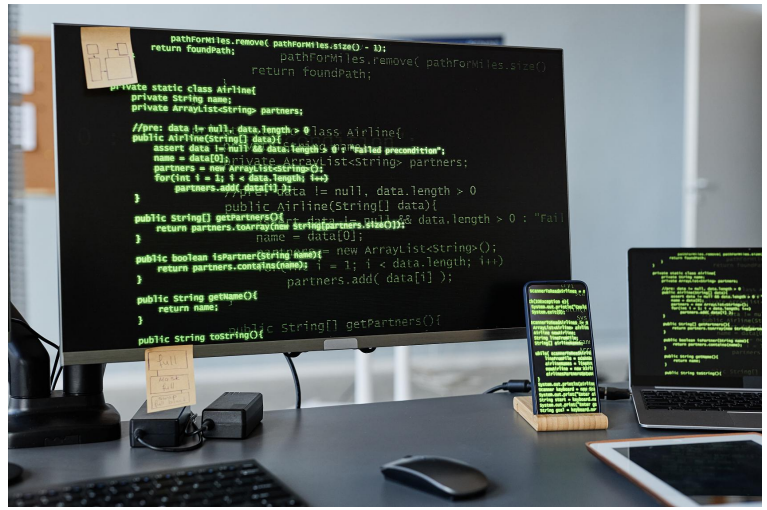
创建一个学生信息表，用来存储学生姓名（姓名作为行键，并且假设姓名不会重复）以及考试成绩，其中，考试成绩是一个列族，分别存储了各个科目的考试成绩。逻辑视图如表所示。

name	score		
	English	Math	Computer

学生信息表的表结构





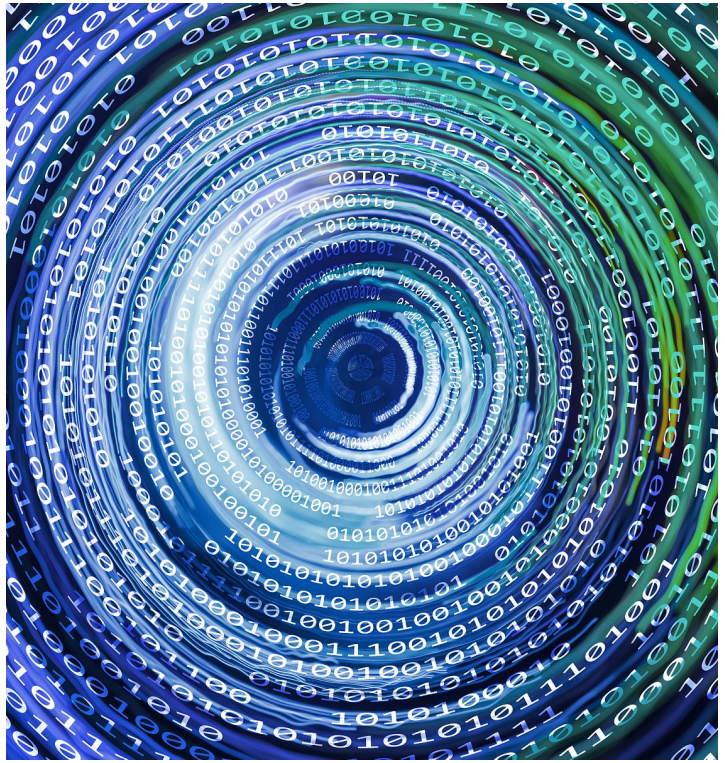


HBase常用Java API及应用实例

■ 创建表

```
/*创建表*/
  /**
   * @param myTableName 表名
   * @param colFamily列族数组
   * @throws Exception
   */
public static void createTable(String myTableName,String[] colFamily) throws IOException {
        TableName tableName = TableName.valueOf(myTableName);
        if(admin.tableExists(tableName)){
            System.out.println("talbe is exists!");
        }else {
            TableDescriptorBuilder tableDescriptor =
TableDescriptorBuilder.newBuilder(tableName);
            for(String str:colFamily){
                ColumnFamilyDescriptor family =
ColumnFamilyDescriptorBuilder.newBuilder(Bytes.toBytes(str)).build();
                tableDescriptor.setColumnFamily(family);
            }
            admin.createTable(tableDescriptor.build());
        }
```

在运行程序时，需要指定参数myTableName为“student”，colFamily为“{“score”}”
程序的运行效果与如下HBase Shell命令等效：
create ‘student’, ‘score’



HBase常用Java API及应用实例

■ 添加数据

现在向表student中添加如下所示的数据。

表中需要添加的数据:

name	score		
	English	Math	Computer
zhangsan	69	86	77
lisi	55	100	88

需要添加的数据







HBase常用Java API及应用实例

■ 添加数据

```
/*添加数据*/
/**
                * @param tableName 表名
                * @param rowKey 行键
                * @param colFamily 列族
                * @param col 列限定符
                * @param val 数据
                * @throws Exception
                */
    public static void insertData(String tableName,String rowKey,String
colFamily,String col,String val) throws IOException {
        Table table = connection.getTable(TableName.valueOf(tableName));
        Put put = new Put(rowKey.getBytes());
        put.addColumn(colFamily.getBytes(),col.getBytes(), val.getBytes());
        table.put(put);
        table.close();
    }
```



HBase常用Java API及应用实例

- **添加数据**

添加数据时，需要分别设置参数myTableName、rowkey、colFamily、col、val的值，然后运行上述代码
例如添加表的第一行数据时，为insertData()方法指定相应参数，并运行如下3行代码：

```
insertData("student","zhangsan","score","English","69");
insertData("student","zhangsan","score","Math","86");
insertData("student","zhangsan","score","Computer","77");
```

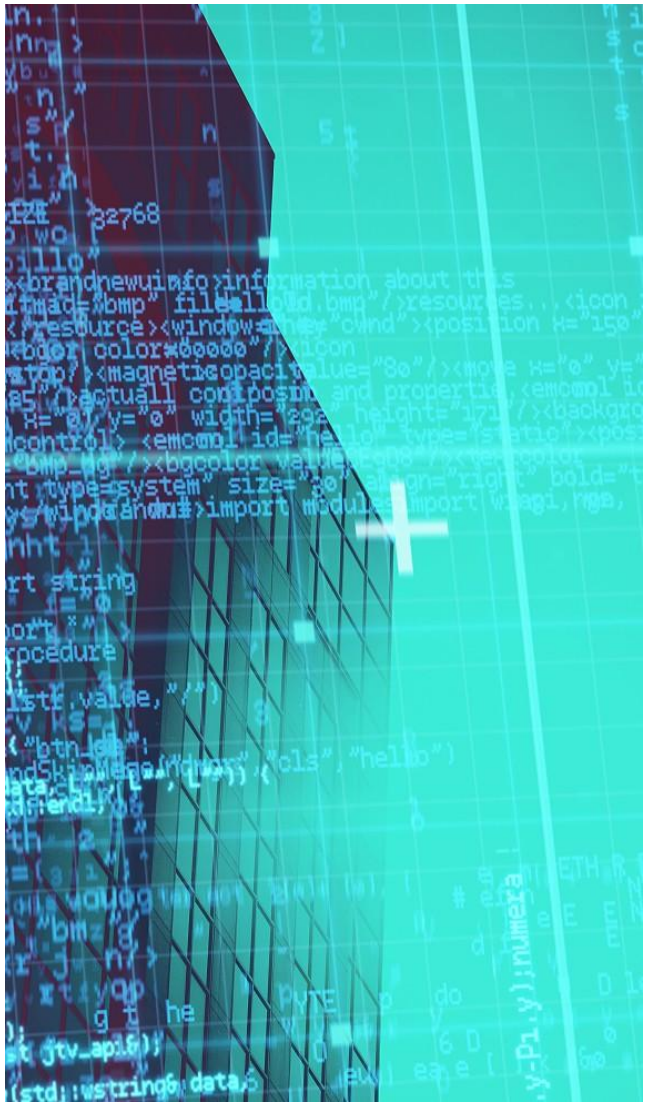
上述代码与如下HBase Shell命令等效：

```
put 'student',‘zhangsan’,‘score:English’,‘69’;
put 'student',‘zhangsan’,‘score:Math’,‘86’;
put 'student',‘zhangsan’,‘score:Computer’,‘77’;
```

HBase常用Java API及应用实例

■ 浏览数据

```
/*获取某单元格数据*/
/**             * @param tableName 表名
                * @param rowKey 行键
                * @param colFamily 列族
                * @param col 列限定符
        * @throws IOException     */
    public static void getData(String tableName,String rowKey,String colFamily, String
col)throws  IOException{
        Table table = connection.getTable(TableName.valueOf(tableName));
        Get get = new Get(rowKey.getBytes());
        get.addColumn(colFamily.getBytes(),col.getBytes());
        Result result = table.get(get);
        System.out.println(new
String(result.getValue(colFamily.getBytes(),col==null?null:col.getBytes())));
        table.close();
    }
```



HBase常用Java API及应用实例

- 浏览数据

比如，现在要获取姓名为“zhangsan”在“English”上的数据，就可以在运行上述代码时，指定参数tableName为“student”、rowKey为“zhangsan”、colFamily为“score”、col为“English”。

```
getData("student", "zhangsan", "score", "English");
```

上述代码与如下HBase Shell命令等效：

```
get ‘student','zhangsan',{COLUMN=>'score:English'}”
```